

Práctica 1: Optimización de Código

Asignatura Tópicos A

Luis Antonio Muñoz Vásquez

Universidad de Magallanes

Mayo de 2026

Facultad de Ingeniería

Departamento de Ingeniería en Computación

Punta Arenas, Chile

Índice

1. Introducción	5
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos específicos	5
3. Marco teórico	6
3.1. Complejidad algorítmica	6
3.1.1. Producto punto	6
3.1.2. Multiplicación matriz-vector	6
3.1.3. Multiplicación matriz-matriz	6
3.2. GNU Octave	7
3.3. CBLAS	7
3.4. Optimización de código	7
4. Metodología	8
4.1. Entorno de trabajo	8
4.2. Procedimiento de medición	8
4.3. Tamaños utilizados	8
5. Implementaciones	8
5.1. Producto punto	8
5.2. Multiplicación matriz-vector	9
5.3. Multiplicación matriz-matriz	10
6. Resultados	11
6.1. Resultados en Octave	11
6.1.1. Producto punto en Octave	11

6.1.2.	Multiplicación matriz-vector en Octave	12
6.1.3.	Multiplicación matriz-matriz en Octave	13
6.2.	Resultados con CBLAS	14
6.2.1.	Producto punto con <code>cblas_ddot</code>	14
6.2.2.	Multiplicación matriz-vector con <code>cblas_dgemv</code>	15
6.2.3.	Multiplicación matriz-matriz con <code>cblas_dgemm</code>	15
6.3.	Comparación C manual, C optimizado y CBLAS	16
6.3.1.	Producto punto	16
6.3.2.	Multiplicación matriz-vector	17
6.3.3.	Multiplicación matriz-matriz	18
7.	Análisis de resultados	19
7.1.	Análisis del producto punto	19
7.2.	Análisis de la multiplicación matriz-vector	19
7.3.	Análisis de la multiplicación matriz-matriz	20
8.	Discusión	20
9.	Conclusiones	21
10.	Anexos	21
10.1.	Estructura de carpetas	21
10.2.	Código principal de producto punto en C	22
10.3.	Código optimizado de producto punto en C	24
10.4.	Código principal de matriz-vector en C	27
10.5.	Código optimizado de matriz-vector en C	29
10.6.	Código principal de matriz-matriz en C	32
10.7.	Código optimizado de matriz-matriz en C	35
10.8.	Script de comparación para producto punto	38

10.9. Script de comparación para matriz-vector 39

10.10Script de comparación para matriz-matriz 40

1. Introducción

La presente práctica tiene como finalidad estudiar el rendimiento computacional de distintas operaciones de álgebra lineal, comparando implementaciones realizadas en GNU Octave, lenguaje C y funciones de la biblioteca CBLAS.

Las operaciones evaluadas fueron el producto punto, la multiplicación matriz-vector y la multiplicación matriz-matriz. Estas operaciones fueron seleccionadas debido a que representan distintos órdenes de complejidad algorítmica: lineal, cuadrático y cúbico, respectivamente.

El trabajo se desarrolló en un entorno WSL, utilizando herramientas de compilación y medición como GCC, Octave, CBLAS, Bash y Gnuplot. Se implementaron versiones manuales en C, versiones optimizadas y se compararon con rutinas especializadas de CBLAS.

El objetivo central fue observar cómo influyen la complejidad algorítmica, el tipo de lenguaje, la optimización de código y la localidad de memoria sobre los tiempos de ejecución.

2. Objetivos

2.1. Objetivo general

Comparar el rendimiento de operaciones de álgebra lineal implementadas en Octave, C manual, C optimizado y CBLAS, evaluando el efecto de la complejidad algorítmica y de las técnicas de optimización de código.

2.2. Objetivos específicos

- Implementar y medir el tiempo de ejecución del producto punto, multiplicación matriz-vector y multiplicación matriz-matriz.
- Comparar operadores nativos de Octave con implementaciones propias.
- Implementar las mismas operaciones en lenguaje C.
- Aplicar técnicas de optimización como loop unrolling y cambio de orden de ciclos.
- Comparar las implementaciones manuales y optimizadas con funciones CBLAS.
- Analizar los resultados obtenidos a partir de tablas y gráficas.

3. Marco teórico

3.1. Complejidad algorítmica

La complejidad algorítmica permite estudiar cómo crece el costo computacional de un algoritmo a medida que aumenta el tamaño del problema. En esta práctica se analizaron tres operaciones fundamentales.

3.1.1. Producto punto

El producto punto entre dos vectores de tamaño n se define como:

$$c = \sum_{i=1}^n x_i y_i \quad (1)$$

La operación requiere recorrer los vectores una vez, por lo que su complejidad es:

$$O(n) \quad (2)$$

3.1.2. Multiplicación matriz-vector

La multiplicación entre una matriz $A \in \mathbb{R}^{n \times n}$ y un vector $x \in \mathbb{R}^n$ se define como:

$$y_i = \sum_{j=1}^n A_{ij} x_j \quad (3)$$

Esta operación requiere dos ciclos anidados, por lo que su complejidad es:

$$O(n^2) \quad (4)$$

3.1.3. Multiplicación matriz-matriz

La multiplicación entre dos matrices cuadradas A y B de tamaño $n \times n$ se define como:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj} \quad (5)$$

Esta operación requiere tres ciclos anidados, por lo que su complejidad es:

$$O(n^3) \tag{6}$$

3.2. GNU Octave

GNU Octave permite resolver operaciones de álgebra lineal mediante operadores nativos. Estos operadores suelen estar asociados a rutinas compiladas y optimizadas, por lo que pueden presentar tiempos de ejecución considerablemente menores que una función escrita directamente en Octave mediante ciclos interpretados.

3.3. CBLAS

CBLAS corresponde a una interfaz en lenguaje C para acceder a rutinas BLAS. Estas rutinas están diseñadas para operaciones de álgebra lineal y suelen estar altamente optimizadas.

En esta práctica se utilizaron las siguientes funciones:

- `cblas_ddot`: producto punto.
- `cblas_dgemv`: multiplicación matriz-vector.
- `cblas_dgemm`: multiplicación matriz-matriz.

3.4. Optimización de código

La optimización de código busca reducir el tiempo de ejecución de un programa sin modificar el resultado final. En esta práctica se aplicaron dos estrategias principales:

- **Loop unrolling**: técnica que reduce la sobrecarga de control de ciclos, realizando varias operaciones por iteración.
- **Cambio de orden de ciclos**: técnica orientada a mejorar el acceso a memoria y la localidad de datos, especialmente relevante en operaciones con matrices.

4. Metodología

4.1. Entorno de trabajo

El desarrollo se realizó en WSL utilizando las siguientes herramientas:

- GNU Octave para implementar y medir operaciones interpretadas.
- GCC para compilar programas en C.
- CBLAS mediante la biblioteca `libblas-dev`.
- Bash para automatizar las mediciones.
- Gnuplot para generar las gráficas.
- LaTeX para elaborar el informe.

4.2. Procedimiento de medición

Para cada tamaño del problema se ejecutó cada algoritmo siete veces. Luego, se calculó el promedio de los tiempos obtenidos. Esta estrategia permite reducir el efecto de variaciones puntuales del sistema operativo o de la carga de CPU.

En Octave se utilizaron las funciones `tic` y `toc`. En C se utilizó la función `gettimeofday`. Los datos fueron almacenados en archivos `.dat` y posteriormente graficados con Gnuplot.

4.3. Tamaños utilizados

Los tamaños de entrada fueron seleccionados de forma progresiva, usando potencias de dos. Se buscó alcanzar tiempos mayores a 0,01 segundos, ya que tiempos demasiado pequeños pueden producir mediciones poco confiables.

5. Implementaciones

5.1. Producto punto

Para el producto punto se implementaron las siguientes versiones:

- Función manual en Octave.
- Operador nativo de Octave.
- Implementación manual en C.
- Implementación optimizada en C mediante loop unrolling.
- Función `cblas_ddot`.

La versión manual en C utiliza un ciclo simple:

```
1 for(i = 0; i < n; i++)
2     c += x[i] * y[i];
```

La versión optimizada realiza varias operaciones dentro de una misma iteración:

```
1 for(i = 0; i < limit; i += 5)
2 {
3     c += x[i] * y[i]
4         + x[i + 1] * y[i + 1]
5         + x[i + 2] * y[i + 2]
6         + x[i + 3] * y[i + 3]
7         + x[i + 4] * y[i + 4];
8 }
```

5.2. Multiplicación matriz-vector

La implementación manual de matriz-vector utiliza dos ciclos anidados:

```
1 for(i = 0; i < m; i++)
2 {
3     y[i] = 0.0;
4     for(j = 0; j < n; j++)
5     {
6         y[i] += A[i * n + j] * x[j];
7     }
8 }
```

La versión optimizada utiliza un acumulador local y loop unrolling en el ciclo interno, disminuyendo accesos innecesarios a memoria.

5.3. Multiplicación matriz-matriz

La implementación manual se realizó mediante tres ciclos anidados en el orden clásico:

```
1  for(i = 0; i < m; i++)
2  {
3      for(j = 0; j < n; j++)
4      {
5          double sum = 0.0;
6          for(p = 0; p < k; p++)
7          {
8              sum += A[i * k + p] * B[p * n + j];
9          }
10         C[i * n + j] = sum;
11     }
12 }
```

La versión optimizada utiliza el orden de ciclos i, p, j , lo que permite recorrer de forma más contigua la matriz B y la matriz C en memoria:

```
1  for(i = 0; i < m; i++)
2  {
3      for(p = 0; p < k; p++)
4      {
5          double a_ip = A[i * k + p];
6          int rowB = p * n;
7          int rowC = i * n;
8
9          for(j = 0; j < n; j++)
10         {
11             C[rowC + j] += a_ip * B[rowB + j];
12         }
13     }
14 }
```

6. Resultados

6.1. Resultados en Octave

6.1.1. Producto punto en Octave

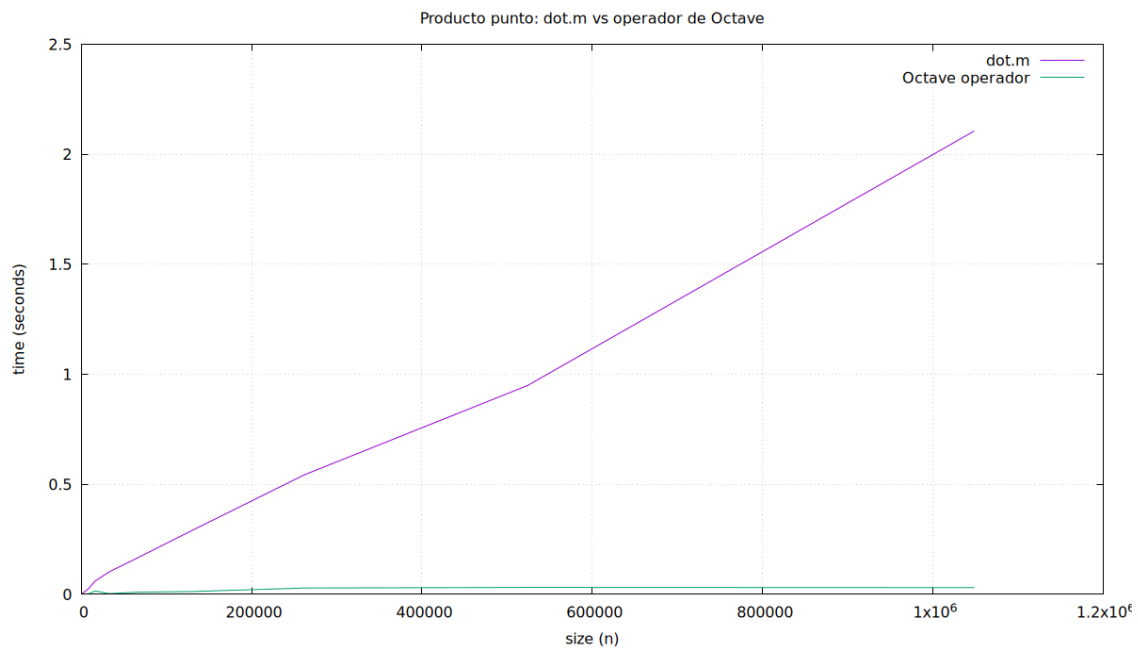


Figura 1: Comparación del producto punto implementado en Octave y operador nativo.

6.1.2. Multiplicación matriz-vector en Octave

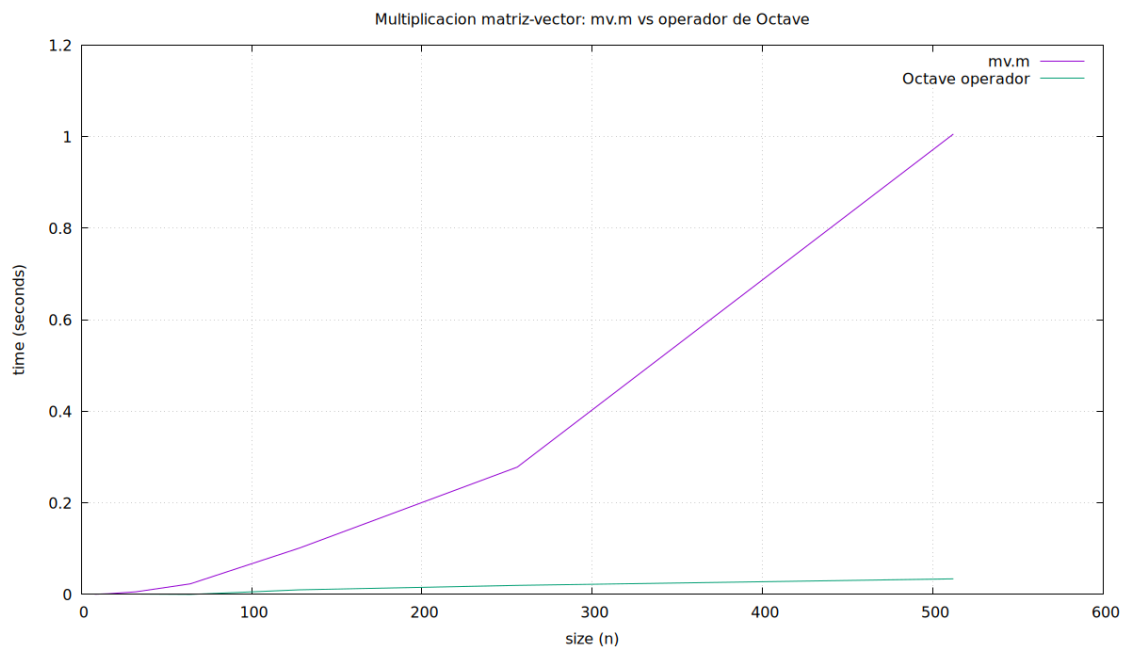


Figura 2: Comparación de multiplicación matriz-vector implementada en Octave y operador nativo.

6.1.3. Multiplicación matriz-matriz en Octave

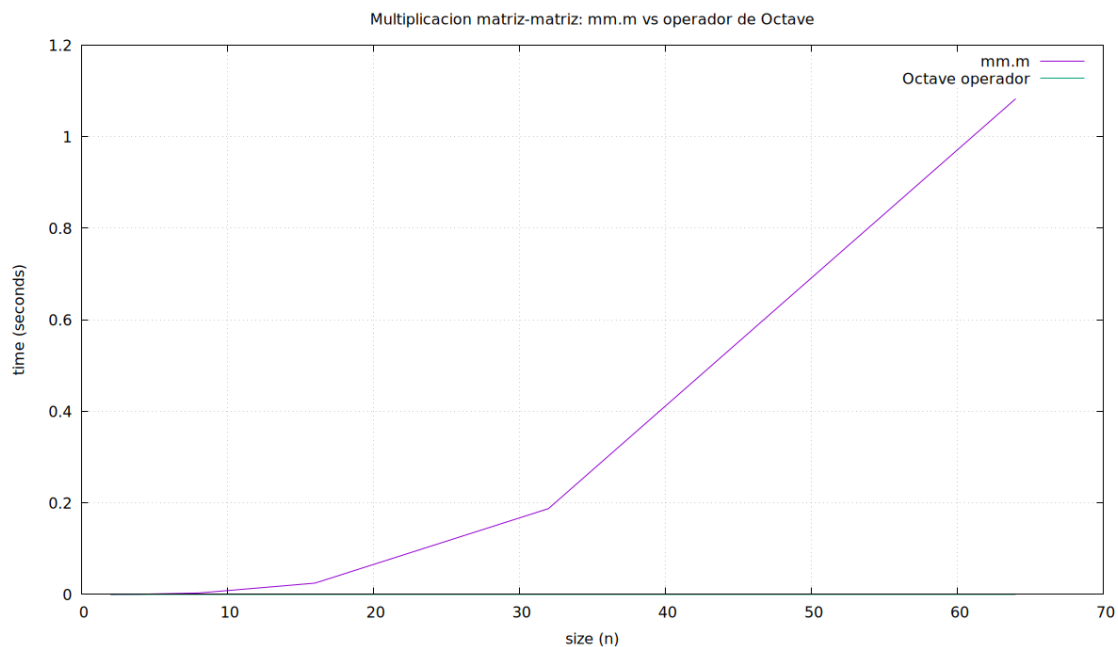


Figura 3: Comparación de multiplicación matriz-matriz implementada en Octave y operador nativo.

6.2. Resultados con CBLAS

6.2.1. Producto punto con cblas_ddot

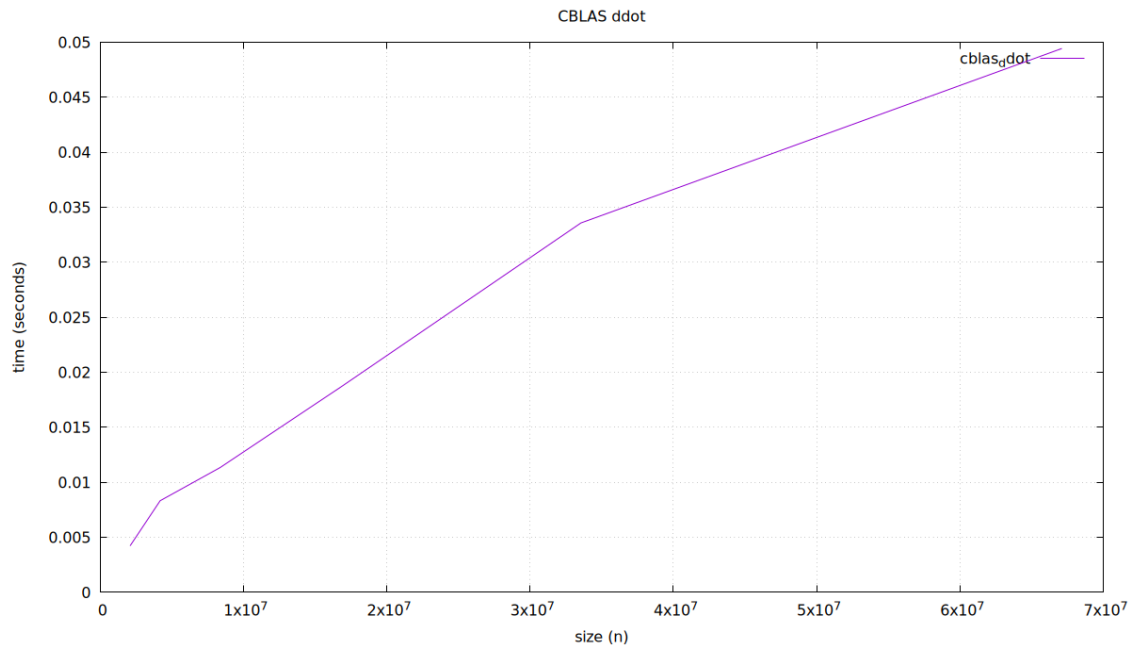


Figura 4: Tiempo de ejecución de `cblas_ddot`.

6.2.2. Multiplicación matriz-vector con `cblas_dgemv`

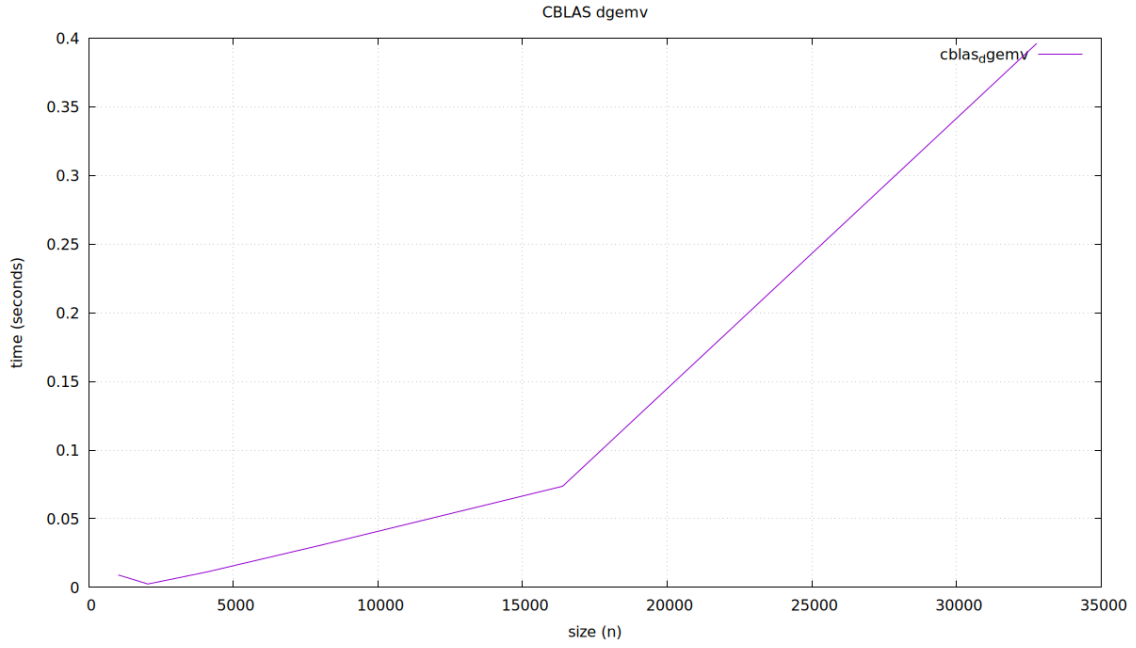


Figura 5: Tiempo de ejecución de `cblas_dgemv`.

6.2.3. Multiplicación matriz-matriz con `cblas_dgemm`

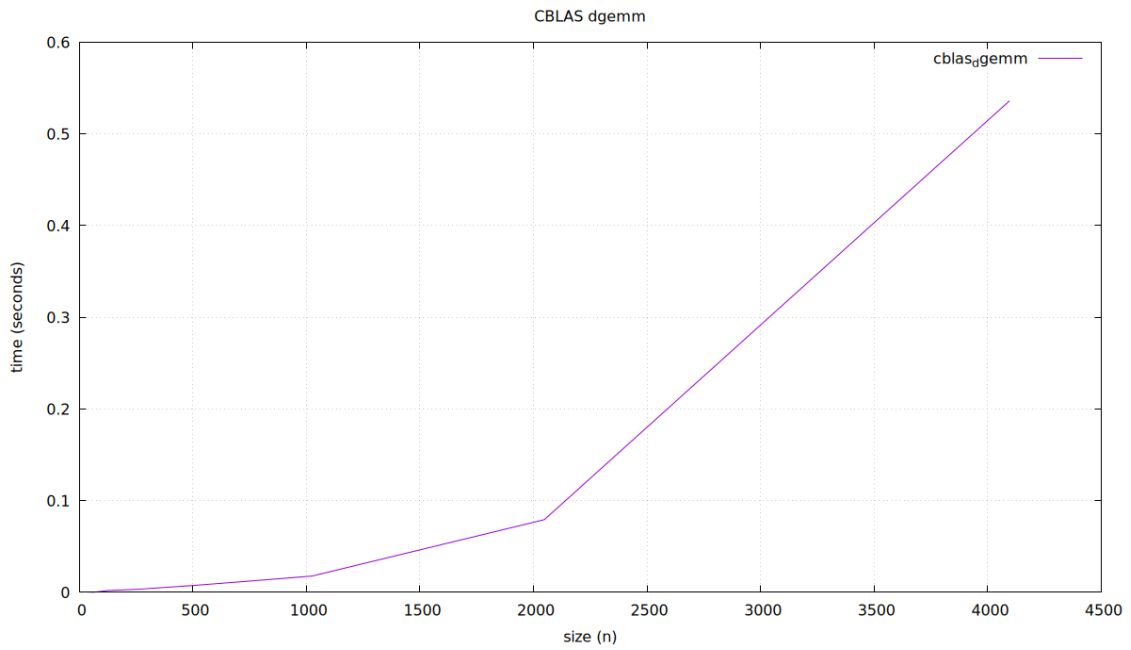


Figura 6: Tiempo de ejecución de `cblas_dgemm`.

6.3. Comparación C manual, C optimizado y CBLAS

6.3.1. Producto punto

Tabla 1: Tiempos de ejecución para producto punto.

n	C manual	C optimizado	CBLAS
2097152	0.00197586	0.00169957	0.00213343
4194304	0.00422486	0.00346971	0.00475314
8388608	0.00798214	0.00614800	0.00580186
16777216	0.01624690	0.01267610	0.00743729
33554432	0.03522290	0.02433260	0.01515530
67108864	0.06923540	0.04843470	0.02964730

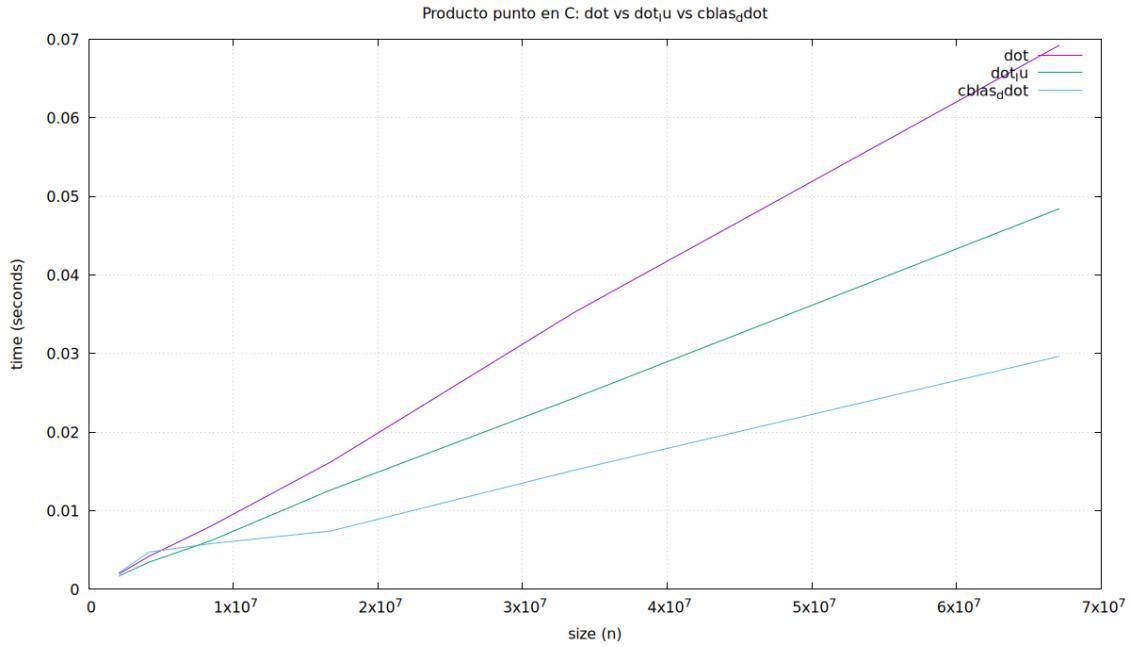


Figura 7: Comparación entre producto punto manual, optimizado y CBLAS.

6.3.2. Multiplicación matriz-vector

Tabla 2: Tiempos de ejecución para multiplicación matriz-vector.

n	C manual	C optimizado	CBLAS
1024	0.00102200	0.00067200	0.00296014
2048	0.00354857	0.00262414	0.00209571
4096	0.01483600	0.00998100	0.00576314
8192	0.05712200	0.03939500	0.01758730
16384	0.05582830	0.15424700	0.06593760
32768	1.12646000	0.99062700	0.36263900

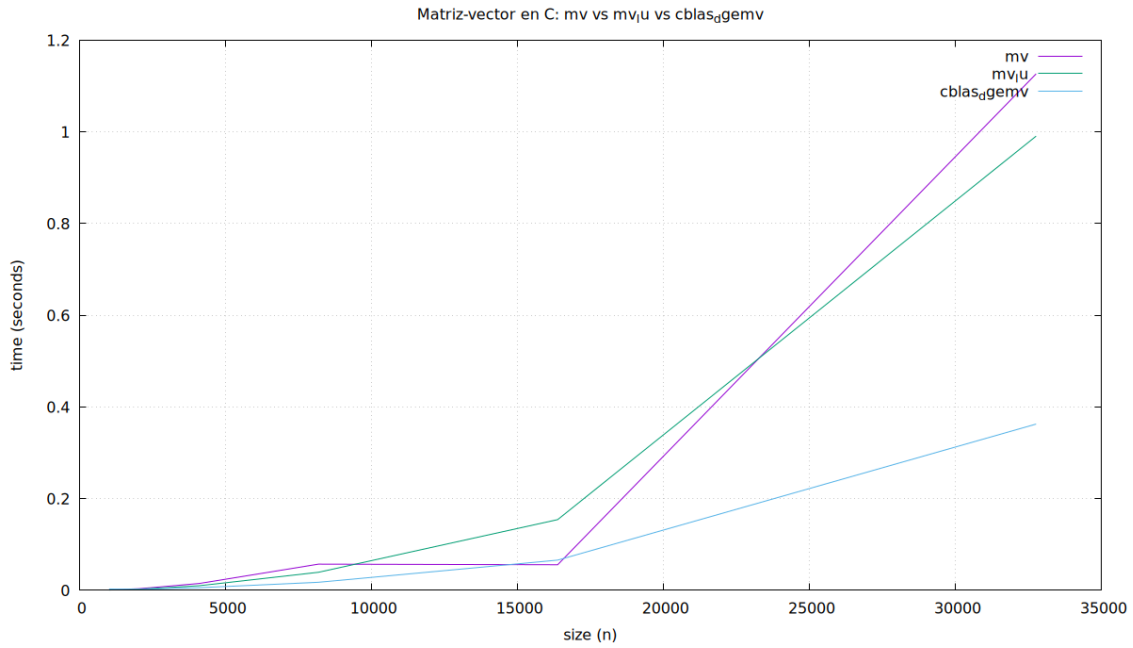


Figura 8: Comparación entre multiplicación matriz-vector manual, optimizada y CBLAS.

6.3.3. Multiplicación matriz-matriz

Tabla 3: Tiempos de ejecución para multiplicación matriz-matriz.

n	C manual	C optimizado	CBLAS
128	0.00263500	0.000505571	0.00109614
256	0.03306990	0.002533000	0.00257829
512	0.81541800	0.018950000	0.00643086
1024	7.14204000	0.162721000	0.02086140
2048	174.15500000	2.759360000	0.06636690
4096	4219.46000000	22.464200000	0.38605000

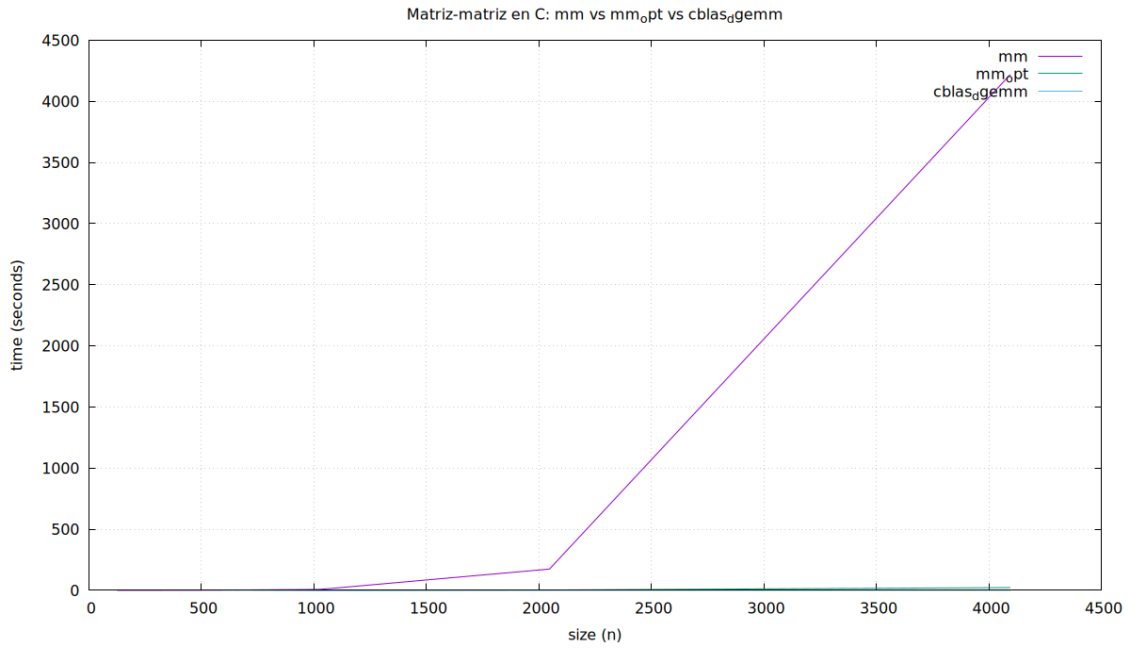


Figura 9: Comparación entre multiplicación matriz-matriz manual, optimizada y CBLAS.

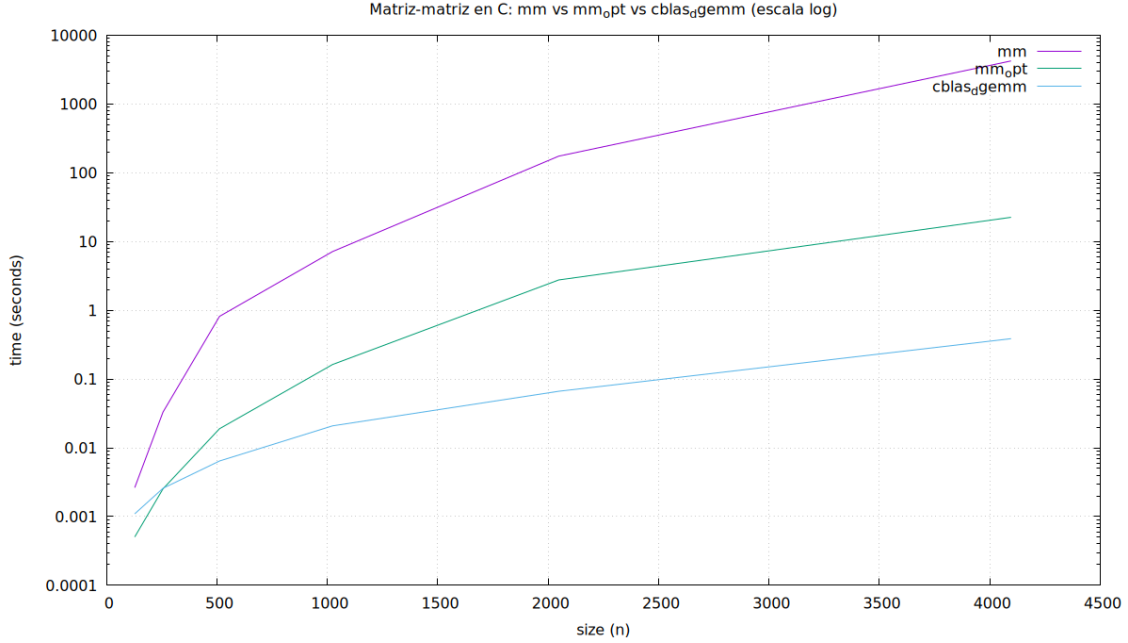


Figura 10: Comparación matriz-matriz en escala logarítmica.

7. Análisis de resultados

7.1. Análisis del producto punto

El producto punto presentó un crecimiento aproximadamente lineal, coherente con su complejidad $O(n)$. En las implementaciones en C se observó que la versión optimizada mediante loop unrolling redujo el tiempo de ejecución en todos los tamaños evaluados.

Para tamaños pequeños, la función `cblas_ddot` no siempre fue más rápida que las versiones manuales. Sin embargo, a medida que aumentó el tamaño del problema, CBLAS presentó mejores resultados. Esto se debe a que las rutinas BLAS están diseñadas para aprovechar mejor las características de la arquitectura y reducir costos de ejecución en problemas de mayor escala.

7.2. Análisis de la multiplicación matriz-vector

La multiplicación matriz-vector presentó un crecimiento superior al producto punto, lo que es coherente con su complejidad $O(n^2)$. La versión optimizada en C logró mejorar el rendimiento respecto a la versión manual en varios tamaños.

Sin embargo, se observó una medición irregular en el tamaño $n = 16384$, donde la versión optimizada presentó un tiempo mayor al esperado. Esta irregularidad puede estar asociada a ruido del sistema, carga del procesador, administración de memoria o efectos de caché. Por esta razón, dicho punto debe ser interpretado como una medición atípica y no como una tendencia general del algoritmo.

En tamaños grandes, CBLAS presentó un rendimiento superior, confirmando la ventaja de utilizar rutinas especializadas para operaciones de álgebra lineal.

7.3. Análisis de la multiplicación matriz-matriz

La multiplicación matriz-matriz fue la operación con mayor diferencia entre las implementaciones. La versión manual presentó tiempos muy elevados, especialmente para tamaños grandes. Esto se explica por su complejidad $O(n^3)$ y por el acceso menos eficiente a memoria.

La versión optimizada mediante cambio del orden de ciclos redujo considerablemente los tiempos de ejecución. En particular, para $n = 4096$, la versión manual tardó 4219.46 segundos, mientras que la versión optimizada tardó 22.4642 segundos. Esto demuestra el impacto de la localidad de memoria sobre el rendimiento.

A pesar de la mejora obtenida, `cblas_dgemm` fue ampliamente superior, alcanzando 0.38605 segundos para el mismo tamaño. Este resultado evidencia el alto nivel de optimización de BLAS para operaciones matriciales.

8. Discusión

Los resultados obtenidos permiten observar que la elección de la implementación influye directamente en el rendimiento. Aunque todos los algoritmos resuelven matemáticamente el mismo problema, sus tiempos de ejecución pueden variar de forma considerable.

En Octave, los operadores nativos fueron mucho más eficientes que las funciones implementadas manualmente con ciclos. Esto se debe a que las funciones escritas directamente en Octave son interpretadas, mientras que los operadores nativos suelen apoyarse en rutinas compiladas.

En C, las optimizaciones aplicadas permitieron mejorar el rendimiento de las versiones manuales. El loop unrolling fue útil en el producto punto y en la multiplicación matriz-vector. Sin embargo, la mejora más significativa ocurrió en la multiplicación matriz-matriz, donde el cambio en el orden de los ciclos permitió mejorar la localidad de memoria.

Finalmente, las funciones CBLAS demostraron el mejor rendimiento general. Esto se debe a que BLAS está diseñado específicamente para operaciones de álgebra lineal y aprovecha optimizaciones internas que son difíciles de igualar con implementaciones manuales simples.

9. Conclusiones

A partir del desarrollo de la práctica, se concluye que la complejidad algorítmica tiene un impacto directo en el tiempo de ejecución. El producto punto presentó un crecimiento lineal, la multiplicación matriz-vector un crecimiento cuadrático y la multiplicación matriz-matriz un crecimiento cúbico.

Las implementaciones en C fueron considerablemente más rápidas que las funciones interpretadas en Octave. Además, las optimizaciones aplicadas permitieron reducir los tiempos de ejecución, especialmente en la multiplicación matriz-matriz.

La técnica de loop unrolling permitió mejorar el rendimiento en operaciones con ciclos internos simples. Por otra parte, el cambio en el orden de los ciclos tuvo un impacto mayor en la multiplicación matriz-matriz debido a la mejora en el acceso a memoria.

CBLAS obtuvo los mejores resultados en la mayoría de los casos, principalmente para tamaños grandes. Esto confirma que, cuando se trabaja con operaciones intensivas de álgebra lineal, es recomendable utilizar bibliotecas optimizadas en lugar de implementaciones manuales.

Finalmente, la práctica permitió comprobar experimentalmente la relación entre complejidad algorítmica, lenguaje de programación, optimización de código, acceso a memoria y rendimiento computacional.

10. Anexos

10.1. Estructura de carpetas

```
1 practica1/  
2 |-- octave/  
3 |-- cblas/  
4 |   |-- array/  
5 |   |-- ddot/  
6 |   |-- dgemv/  
7 |   |-- dgemm/
```

```

8 | -- c/
9 |   | -- dot/
10 |   | -- mv/
11 |   | -- mm/
12 | -- resultados/
13 |   | -- datos/
14 |   | -- graficos/
15 | -- informe_latex/

```

10.2. Código principal de producto punto en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static double my_dot(const int n, const double* x, const double* y);
16 static int main_dot(const int n, const int elements_type, const char
    * verbose);
17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "dot: manual C program for dot product.\n");
21     fputc('\n', stdout);
22
23     if(argc != 4)
24     {
25         fprintf(stdout, "Use: dot <n:int> <0|1|2> <on|off>.\n");
26         return EXIT_FAILURE;

```

```

27     }
28
29     main_dot(atoi(argv[1]), atoi(argv[2]), argv[3]);
30     return EXIT_SUCCESS;
31 }
32
33 static double my_dot(const int n, const double* x, const double* y)
34 {
35     double c = 0.0;
36     int i = 0;
37
38     for(i = 0; i < n; i++)
39         c += x[i] * y[i];
40
41     return c;
42 }
43
44 static int main_dot(const int n, const int elements_type, const char
    * verbose)
45 {
46     double c = 0.0;
47     double* x = NULL;
48     double* y = NULL;
49     struct timeval start, finish;
50     double runtime = 0.0;
51
52     assert(n > 0);
53     assert(elements_type >= ZEROS && elements_type <= RAND);
54
55     x = array_new(n, 1, elements_type);
56     assert(x != NULL);
57
58     y = array_new(n, 1, elements_type);
59     assert(y != NULL);
60
61     gettimeofday(&start, NULL);
62     c = my_dot(n, x, y);
63     gettimeofday(&finish, NULL);

```

```

64
65     if(strcmp(verbose, "on") == 0)
66     {
67         array_show(n, 1, x, "x");
68         array_show(n, 1, y, "y");
69     }
70
71     fprintf(stdout, "c = %lf\n", c);
72     runtime = timeval_diff(&finish, &start);
73     fprintf(stdout, "Data: %d %lf\n", n, runtime);
74
75     free(x);
76     free(y);
77
78     return EXIT_SUCCESS;
79 }

```

10.3. Código optimizado de producto punto en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static double my_dot_lu(const int n, const double* x, const double*
    y);
16 static int main_dot_lu(const int n, const int elements_type, const
    char* verbose);
17

```



```

18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "dot_lu: manual C program for dot product with
21         loop unrolling.\n");
22     fputc('\n', stdout);
23
24     if(argc != 4)
25     {
26         fprintf(stdout, "Use: dot_lu <n:int> <0|1|2> <on|off>.\n");
27         return EXIT_FAILURE;
28     }
29
30     main_dot_lu(atoi(argv[1]), atoi(argv[2]), argv[3]);
31     return EXIT_SUCCESS;
32 }
33
34 static double my_dot_lu(const int n, const double* x, const double*
35     y)
36 {
37     double c = 0.0;
38     int i = 0;
39     int limit = n - (n % 5);
40
41     for(i = 0; i < limit; i += 5)
42     {
43         c += x[i] * y[i]
44             + x[i + 1] * y[i + 1]
45             + x[i + 2] * y[i + 2]
46             + x[i + 3] * y[i + 3]
47             + x[i + 4] * y[i + 4];
48     }
49
50     for(i = limit; i < n; i++)
51     {
52         c += x[i] * y[i];
53     }
54
55     return c;

```

```

54 }
55
56 static int main_dot_lu(const int n, const int elements_type, const
    char* verbose)
57 {
58     double c = 0.0;
59     double* x = NULL;
60     double* y = NULL;
61     struct timeval start, finish;
62     double runtime = 0.0;
63
64     assert(n > 0);
65     assert(elements_type >= ZEROS && elements_type <= RAND);
66
67     x = array_new(n, 1, elements_type);
68     assert(x != NULL);
69
70     y = array_new(n, 1, elements_type);
71     assert(y != NULL);
72
73     gettimeofday(&start, NULL);
74     c = my_dot_lu(n, x, y);
75     gettimeofday(&finish, NULL);
76
77     if(strcmp(verbose, "on") == 0)
78     {
79         array_show(n, 1, x, "x");
80         array_show(n, 1, y, "y");
81     }
82
83     fprintf(stdout, "c = %lf\n", c);
84     runtime = timeval_diff(&finish, &start);
85     fprintf(stdout, "Data: %d %lf\n", n, runtime);
86
87     free(x);
88     free(y);
89
90     return EXIT_SUCCESS;

```

```
91 }
```

10.4. Código principal de matriz-vector en C

```
1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static void my_mv(const int m, const int n, const double* A, const
    double* x, double* y);
16 static int main_mv(const int m, const int n, const int elements_type
    , const char* verbose);
17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "mv: manual C program for matrix-vector
        multiplication.\n");
21     fputc('\n', stdout);
22
23     if(argc != 5)
24     {
25         fprintf(stdout, "Use: mv <m:int> <n:int> <0|1|2> <on|off>.\n
            ");
26         return EXIT_FAILURE;
27     }
28
29     main_mv(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), argv[4]);
30     return EXIT_SUCCESS;
```

```

31 }
32
33 static void my_mv(const int m, const int n, const double* A, const
    double* x, double* y)
34 {
35     int i, j;
36
37     for(i = 0; i < m; i++)
38     {
39         y[i] = 0.0;
40         for(j = 0; j < n; j++)
41         {
42             y[i] += A[i * n + j] * x[j];
43         }
44     }
45 }
46
47 static int main_mv(const int m, const int n, const int elements_type
    , const char* verbose)
48 {
49     double* A = NULL;
50     double* x = NULL;
51     double* y = NULL;
52     struct timeval start, finish;
53     double runtime = 0.0;
54
55     assert(m > 0);
56     assert(n > 0);
57     assert(elements_type >= ZEROS && elements_type <= RAND);
58
59     A = array_new(m, n, elements_type);
60     assert(A != NULL);
61
62     x = array_new(n, 1, elements_type);
63     assert(x != NULL);
64
65     y = array_new(m, 1, ZEROS);
66     assert(y != NULL);

```

```

67
68     gettimeofday(&start, NULL);
69     my_mv(m, n, A, x, y);
70     gettimeofday(&finish, NULL);
71
72     if(strcmp(verbose, "on") == 0)
73     {
74         array_show(m, n, A, "A");
75         array_show(n, 1, x, "x");
76     }
77
78     array_show(m, 1, y, "y");
79     runtime = timeval_diff(&finish, &start);
80     fprintf(stdout, "Data: %d %lf\n", m, runtime);
81
82     free(A);
83     free(x);
84     free(y);
85
86     return EXIT_SUCCESS;
87 }

```

10.5. Código optimizado de matriz-vector en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14

```

```

15 static void my_mv_lu(const int m, const int n, const double* A,
    const double* x, double* y);
16 static int main_mv_lu(const int m, const int n, const int
    elements_type, const char* verbose);
17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "mv_lu: manual C program for matrix-vector
        multiplication with loop unrolling.\n");
21     fputc('\n', stdout);
22
23     if(argc != 5)
24     {
25         fprintf(stdout, "Use: mv_lu <m:int> <n:int> <0|1|2> <on|off
            >.\n");
26         return EXIT_FAILURE;
27     }
28
29     main_mv_lu(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), argv[4])
        ;
30     return EXIT_SUCCESS;
31 }
32
33 static void my_mv_lu(const int m, const int n, const double* A,
    const double* x, double* y)
34 {
35     int i, j;
36     int limit = n - (n % 4);
37
38     for(i = 0; i < m; i++)
39     {
40         double sum = 0.0;
41         int base = i * n;
42
43         for(j = 0; j < limit; j += 4)
44         {
45             sum += A[base + j] * x[j]
46                 + A[base + j + 1] * x[j + 1]

```

```

47         + A[base + j + 2] * x[j + 2]
48         + A[base + j + 3] * x[j + 3];
49     }
50
51     for(j = limit; j < n; j++)
52     {
53         sum += A[base + j] * x[j];
54     }
55
56     y[i] = sum;
57 }
58 }
59
60 static int main_mv_lu(const int m, const int n, const int
    elements_type, const char* verbose)
61 {
62     double* A = NULL;
63     double* x = NULL;
64     double* y = NULL;
65     struct timeval start, finish;
66     double runtime = 0.0;
67
68     assert(m > 0);
69     assert(n > 0);
70     assert(elements_type >= ZEROS && elements_type <= RAND);
71
72     A = array_new(m, n, elements_type);
73     assert(A != NULL);
74
75     x = array_new(n, 1, elements_type);
76     assert(x != NULL);
77
78     y = array_new(m, 1, ZEROS);
79     assert(y != NULL);
80
81     gettimeofday(&start, NULL);
82     my_mv_lu(m, n, A, x, y);
83     gettimeofday(&finish, NULL);

```

```

84
85     if(strcmp(verbose, "on") == 0)
86     {
87         array_show(m, n, A, "A");
88         array_show(n, 1, x, "x");
89     }
90
91     array_show(m, 1, y, "y");
92     runtime = timeval_diff(&finish, &start);
93     fprintf(stdout, "Data: %d %lf\n", m, runtime);
94
95     free(A);
96     free(x);
97     free(y);
98
99     return EXIT_SUCCESS;
100 }

```

10.6. Código principal de matriz-matriz en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static void my_mm(const int m, const int n, const int k, const
    double* A, const double* B, double* C);
16 static int main_mm(const int m, const int n, const int k, const int
    elements_type, const char* verbose);

```



```

17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "mm: manual C program for matrix-matrix
21         multiplication.\n");
22     fputc('\n', stdout);
23
24     if(argc != 6)
25     {
26         fprintf(stdout, "Use: mm <m:int> <n:int> <k:int> <0|1|2> <on
27             |off>.\n");
28         return EXIT_FAILURE;
29     }
30
31     main_mm(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), atoi(argv
32         [4]), argv[5]);
33     return EXIT_SUCCESS;
34 }
35
36 static void my_mm(const int m, const int n, const int k, const
37     double* A, const double* B, double* C)
38 {
39     int i, j, p;
40
41     for(i = 0; i < m; i++)
42     {
43         for(j = 0; j < n; j++)
44         {
45             double sum = 0.0;
46             for(p = 0; p < k; p++)
47             {
48                 sum += A[i * k + p] * B[p * n + j];
49             }
50             C[i * n + j] = sum;
51         }
52     }
53 }

```

```

51 static int main_mm(const int m, const int n, const int k, const int
    elements_type, const char* verbose)
52 {
53     double* A = NULL;
54     double* B = NULL;
55     double* C = NULL;
56     struct timeval start, finish;
57     double runtime = 0.0;
58
59     assert(m > 0);
60     assert(n > 0);
61     assert(k > 0);
62     assert(elements_type >= ZEROS && elements_type <= RAND);
63
64     A = array_new(m, k, elements_type);
65     assert(A != NULL);
66
67     B = array_new(k, n, elements_type);
68     assert(B != NULL);
69
70     C = array_new(m, n, ZEROS);
71     assert(C != NULL);
72
73     gettimeofday(&start, NULL);
74     my_mm(m, n, k, A, B, C);
75     gettimeofday(&finish, NULL);
76
77     if(strcmp(verbose, "on") == 0)
78     {
79         array_show(m, k, A, "A");
80         array_show(k, n, B, "B");
81     }
82
83     array_show(m, n, C, "C");
84     runtime = timeval_diff(&finish, &start);
85     fprintf(stdout, "Data: %d %lf\n", m, runtime);
86
87     free(A);

```

```

88     free(B);
89     free(C);
90
91     return EXIT_SUCCESS;
92 }

```

10.7. Código optimizado de matriz-matriz en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static void my_mm_opt(const int m, const int n, const int k, const
    double* A, const double* B, double* C);
16 static int main_mm_opt(const int m, const int n, const int k, const
    int elements_type, const char* verbose);
17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "mm_opt: optimized C program for matrix-matrix
        multiplication.\n");
21     fputc('\n', stdout);
22
23     if(argc != 6)
24     {
25         fprintf(stdout, "Use: mm_opt <m:int> <n:int> <k:int> <0|1|2>
            <on|off>.\n");
26         return EXIT_FAILURE;

```

```

27     }
28
29     main_mm_opt(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), atoi(
        argv[4]), argv[5]);
30     return EXIT_SUCCESS;
31 }
32
33 static void my_mm_opt(const int m, const int n, const int k, const
    double* A, const double* B, double* C)
34 {
35     int i, j, p;
36
37     for(i = 0; i < m; i++)
38     {
39         for(j = 0; j < n; j++)
40             C[i * n + j] = 0.0;
41     }
42
43     for(i = 0; i < m; i++)
44     {
45         for(p = 0; p < k; p++)
46         {
47             double a_ip = A[i * k + p];
48             int rowB = p * n;
49             int rowC = i * n;
50
51             for(j = 0; j < n; j++)
52             {
53                 C[rowC + j] += a_ip * B[rowB + j];
54             }
55         }
56     }
57 }
58
59 static int main_mm_opt(const int m, const int n, const int k, const
    int elements_type, const char* verbose)
60 {
61     double* A = NULL;

```

```

62     double* B = NULL;
63     double* C = NULL;
64     struct timeval start, finish;
65     double runtime = 0.0;
66
67     assert(m > 0);
68     assert(n > 0);
69     assert(k > 0);
70     assert(elements_type >= ZEROS && elements_type <= RAND);
71
72     A = array_new(m, k, elements_type);
73     assert(A != NULL);
74
75     B = array_new(k, n, elements_type);
76     assert(B != NULL);
77
78     C = array_new(m, n, ZEROS);
79     assert(C != NULL);
80
81     gettimeofday(&start, NULL);
82     my_mm_opt(m, n, k, A, B, C);
83     gettimeofday(&finish, NULL);
84
85     if(strcmp(verbose, "on") == 0)
86     {
87         array_show(m, k, A, "A");
88         array_show(k, n, B, "B");
89     }
90
91     array_show(m, n, C, "C");
92     runtime = timeval_diff(&finish, &start);
93     fprintf(stdout, "Data: %d %lf\n", m, runtime);
94
95     free(A);
96     free(B);
97     free(C);
98
99     return EXIT_SUCCESS;

```

```
100 }
```

10.8. Script de comparación para producto punto

```
1  #!/bin/bash
2
3  i=0
4  vals[0]=2097152
5
6  while [ $i -lt 5 ]
7  do
8      valtmp=${vals[$i]}
9      newval=$(echo "$valtmp * 2" | bc)
10     i=$((i + 1))
11     vals[$i]=$newval
12 done
13
14 unset LC_ALL
15 export LC_NUMERIC=C
16
17 echo "" > dot_compare.dat
18
19 for val in "${vals[@]}"
20 do
21     echo "" > data_dot.dat
22     echo "" > data_dot_lu.dat
23     echo "" > data_cblas.dat
24
25     for i in {1..7}
26     do
27         ./dot "${val}" 1 off | awk '/^Data/ { print $2 " " $3 }' >>
            data_dot.dat
28         ./dot_lu "${val}" 1 off | awk '/^Data/ { print $2 " " $3 }'
            >> data_dot_lu.dat
29         ~/practica1/cblas/ddot/ddot "${val}" 1 off | awk '/^Data/ {
            print $2 " " $3 }' >> data_cblas.dat
30         echo "${val} ${i} test finished"
31     done
```

```

32
33     t_dot=$(awk '{sum += $2} END {print sum/7}' data_dot.dat)
34     t_dot_lu=$(awk '{sum += $2} END {print sum/7}' data_dot_lu.dat)
35     t_cblas=$(awk '{sum += $2} END {print sum/7}' data_cblas.dat)
36
37     echo "${val} ${t_dot} ${t_dot_lu} ${t_cblas}" >> dot_compare.dat
38 done
39
40 awk 'NF > 0' dot_compare.dat

```

10.9. Script de comparación para matriz-vector

```

1  #!/bin/bash
2
3  i=0
4  vals[0]=1024
5
6  while [ $i -lt 5 ]
7  do
8      valtmp=${vals[$i]}
9      newval=$(echo "$valtmp * 2" | bc)
10     i=$((i + 1))
11     vals[$i]=$newval
12 done
13
14 unset LC_ALL
15 export LC_NUMERIC=C
16
17 echo "" > mv_compare.dat
18
19 for val in "${vals[@]}"
20 do
21     echo "" > data_mv.dat
22     echo "" > data_mv_lu.dat
23     echo "" > data_cblas.dat
24
25     for i in {1..7}
26     do

```

```

27     ./mv "${val}" "${val}" 1 off | awk '/^Data/ { print $2 " "
        $3 }' >> data_mv.dat
28     ./mv_lu "${val}" "${val}" 1 off | awk '/^Data/ { print $2 "
        " $3 }' >> data_mv_lu.dat
29     ~/practical/cblas/dgemv/dgemv "${val}" "${val}" 1 off | awk
        '/^Data/ { print $2 " " $3 }' >> data_cblas.dat
30     echo "${val} ${i} test finished"
31 done
32
33     t_mv=$(awk '{sum += $2} END {print sum/7}' data_mv.dat)
34     t_mv_lu=$(awk '{sum += $2} END {print sum/7}' data_mv_lu.dat)
35     t_cblas=$(awk '{sum += $2} END {print sum/7}' data_cblas.dat)
36
37     echo "${val} ${t_mv} ${t_mv_lu} ${t_cblas}" >> mv_compare.dat
38 done
39
40 awk 'NF > 0' mv_compare.dat

```

10.10. Script de comparación para matriz-matriz

```

1  #!/bin/bash
2
3  i=0
4  vals[0]=128
5
6  while [ $i -lt 5 ]
7  do
8      valtmp=${vals[$i]}
9      newval=$(echo "$valtmp * 2" | bc)
10     i=$((i + 1))
11     vals[$i]=$newval
12 done
13
14 unset LC_ALL
15 export LC_NUMERIC=C
16
17 echo "" > mm_compare.dat
18

```



```

19 for val in "${vals[@]}"
20 do
21     echo "" > data_mm.dat
22     echo "" > data_mm_opt.dat
23     echo "" > data_cblas.dat
24
25     for i in {1..7}
26     do
27         ./mm "${val}" "${val}" "${val}" 1 off | awk '/^Data/ { print
28             $2 " " " $3 }' >> data_mm.dat
29         ./mm_opt "${val}" "${val}" "${val}" 1 off | awk '/^Data/ {
30             print $2 " " " $3 }' >> data_mm_opt.dat
31         ~/practical1/cblas/dgemm/dgemm "${val}" "${val}" "${val}" 1
32             off | awk '/^Data/ { print $2 " " " $3 }' >> data_cblas.dat
33         echo "${val} ${i} test finished"
34     done
35
36     t_mm=$(awk '{sum += $2} END {print sum/7}' data_mm.dat)
37     t_mm_opt=$(awk '{sum += $2} END {print sum/7}' data_mm_opt.dat)
38     t_cblas=$(awk '{sum += $2} END {print sum/7}' data_cblas.dat)
39
40     echo "${val} ${t_mm} ${t_mm_opt} ${t_cblas}" >> mm_compare.dat
41 done
42
43 awk 'NF > 0' mm_compare.dat

```